

Lecture Notes, Fast

A practical LaTeX + VimTeX desk reference

The 10-second loop

Type notes with snippets and the short formatting triggers. Let the 650 ms save debounce do its work. When you want a checkpoint: **Space la** saves and compiles; **Space lv** opens the PDF; **Space le** shows errors.

Your key language

Key	Action
Space	leader key for commands
jk	leave Insert mode quickly
Space w	save current file
Space q	quit window
Ctrl-h/j/k/l	move between splits
Space zw	distraction-free writing

Compile and inspect

Keys	Use
Space ll	start/refresh continuous compile
Space la	save, then compile
Space lv	view PDF in Skim
Space le	inspect compiler errors
Space lt	toggle table of contents
Space lk	stop compiler
Space lm	one-shot compile
Space lo	show compiler output
Space lc	clean auxiliary files
Space li	show VimTeX project info
Space ts	toggle spell checking

Quickfix does not open automatically.

Use **Space le** only when you want diagnostics, so your cursor and page stay stable while lecturing.

Find things without leaving the file

Space ff	find a file
Space fg	search text across the project
Space fs	jump through document symbols
Space fb	choose an open buffer
Space e	toggle the file tree

The fast lecture pattern

1. Open or create a ‘tex’ file. A new file receives your template automatically.
2. Type a trigger such as **defn**, **thm**, **proof**, **al**, or **eq**; expand it with your completion key (usually **Tab**).
3. Use **;c** for a fresh italic Claim paragraph, **;p** for Proof, and **;r** for Remark. The **claim** snippet adds a Claim plus a matched proof in one go.
4. Press **Space la** at a natural pause. If something breaks, press **Space le**.

Formatting while you type

One-line emphasis

In Insert mode:

;b	inserts <code>\textbf{}</code> and places the cursor inside.
;i	inserts <code>\emph{}</code> and places the cursor inside.
;c	starts a new paragraph labeled <i>Claim</i> .
;p	starts a new paragraph labeled <i>Proof</i> .
;r	starts a new paragraph labeled <i>Remark</i> .
\(	inserts <code>\(</code> with the cursor inside.
\[	inserts <code>\[</code> with the cursor inside.

LuaSnip triggers already installed

Statements and proof structure

prb	problem plus solution
prob	numbered problem
hw	<code>\exercise{#.#}</code> header + solution (solutions.tex)
hwp	auto-numbered Problem N header + solution (homework files)
chb	chapter banner (solutions.tex)
sol, ans	solution or answer block
claim	claim + proof pair (italic labels, auto <code>\qed</code>)
defn	definition
thm, lem, prop, cor	theorem, lemma, proposition, corollary
ex, rem, proof	example, remark, proof

Display and layout

al, eq	align or equation display
cases, mat	cases or matrix
enum, item	numbered or bulleted list
sec, ssec	section or subsection

Mathematical shorthand

rr, cc, qq, zz, nn	$\mathbb{R}, \mathbb{C}, \mathbb{Q}, \mathbb{Z}, \mathbb{N}$
abs, norm, ip, set	absolute value, norm, inner product, set
span, rank, ker, im	common linear/algebra operators
lim, int, pd, dv	limits, integrals, partial/ordinary derivatives
pset, cond, ev, var, cov	probability notation

Good source habits

- Each course folder keeps one growing `solutions.tex` with a banner per textbook chapter. Jump to the chapter (Toc: `Space 1t`), type `hw`, press `Tab`, and write.
- Put one mathematical claim per paragraph. Start it with `;c`.
- Use `align*` for multiple equalities; put `'&'` before the alignment point and `'` between lines.
- Use `\text{...}` inside math for words: $x > 0$ if $x \neq 0$.
- Label important results: `\label{thm:name}`; reference them with `\cref{thm:name}`.
- Keep scratch explanations in `\textbf{Claim.}`, `{Proof.}`, or `{Idea.}` rather than burying them in comments.

Useful emergency commands

<code>:VimtexCompile</code>	start/refresh compile
<code>:VimtexView</code>	open the PDF
<code>:VimtexErrors</code>	inspect errors
<code>:VimtexTocToggle</code>	show/hide outline
<code>:VimtexStop</code>	stop compilation
<code>:VimtexInfo</code>	inspect project configuration

Remember

If a shortcut is forgotten: press `Space` and wait for which-key/help prompts, or use `:h vimtex`.